

C Sales Orders

From Sage Evolution SDK | Documentation Portal

The SDK only Allows for Invoice documents to be processed through SalesOrder class. Sales Orders can either be placed by using the Save() method or they can be processed in to a Invoice using the Process() method. When a order is placed it is just a request for goods and is not posted to any posting tables but gets saved as a document in the invnum table and affects the items quantities ordered. Once the order is processed the necessary posting tables are populated affecting the General Ledger, Customer Ledger and Inventory. Since the ordered quantity might not be the Sold quantities , orders can be partially processed. A Sales Quotation can also be done as well as reserving of stock.

The following example displays how a typical order is placed using the SDK thereafter the order is partially processed then the balance of the order is completed:

```
//Create a new instance of a SalesOrder class
SalesOrder order = new SalesOrder();
Customer cust = new Customer("Customer1");
order.Customer = cust; //Assign a value to the Customer property
//the Detail property contains the document lines
order.Detail.Add("ItemA", 5, 10);
order.Save(); //Place the Order
string orderNo = order.OrderNo;
MessageBox.Show(orderNo);
```

There are a number of overloaded **Add** methods. The **Add** method used above specifies the product code as a string, but you may well find that the other versions of this method will better suit your purpose. Assuming the account is taxable, the document total as well as the total for line 1 (index 0 in the collection) will result in 57.00, local currency. Every sales order must have an order number, so at this point, unless you are supplying your own order number, you may want to store the value of **order.OrderNo**, which is generated.

To process the Order in to a Invoice we would need to specify a quantity to process and use the process method as follows.

```
order.Detail[0].ToProcess = 2;
order.Process();
```

If we set the **ToProcess** quantity to 5, we would in fact be completing the order when we call Process. The Process function takes care of all the underlying transactions and returns the reference number of the invoice transaction created. Thereafter there would be no point in keeping this order opened, since the quantity of stock requested has now been sold; hence the order is then marked as archived and cannot be processed further. By setting the **ToProcess** quantity above to 2, you would end up with 2 documents: an archived document for 2 units, and a partially processed document for the remaining 3 units. The archived document cannot be altered, but we can later retrieve the partially processed document. Bys using the complete method we can even process the total quantity outstanding automatically and not specifying a quantity as below

```
SalesOrder newOrder = new SalesOrder(orderNo);
newOrder.Complete(); // the complete method completes the whole unprocessed order
```

The following example displays a Sales order using a GL line as well as a inventory item with the customers default price list also we specify addresses to be used. To use a GL Account as a item the Account in Evolution has to first be edited and flagged to be used on a order:

```

SalesOrder SO = new SalesOrder();
SO.Customer = new Customer("CustomerSDK1");
SO.InvoiceDate = DateTime.Now; // choose to set the invoice date or Order date etc
SO.InvoiceTo = SO.Customer.PostalAddress.Condense(); //InvoiceTo will be the postal address Condense method can be used
SO.DeliverTo = new Address("Physical Address 1", "Address 2", "Address 3", "Address 4", "Address 5", "PC"); //DeliverTo will be the delivery address
SO.Project = new Project("P1"); //Various SO properties like project can be set

OrderDetail OD = new OrderDetail();
SO.Detail.Add(OD);
//Various Order Detail properties can be added like warehouse , sales reps , userfields etc
OD.InventoryItem = new InventoryItem("ItemA"); //Use the inventoryItem constructor to specify a Item
OD.Quantity = 10;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = OD.InventoryItem.SellingPrices[SO.Customer.DefaultPriceList].PriceIncl; //Specify the Customer Price List

OD = new OrderDetail();
SO.Detail.Add(OD);
OD.GLAccount = new GLAccount("Accounting Fees"); //Use the GLAccount Item constructor to specify a Account
OD.Quantity = 1;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 30;

SO.Process();

```

The following example is on processing a Sales Order Quotation:

```

SalesOrderQuotation SOQ = new SalesOrderQuotation();
SOQ.Customer = new Customer("Cust1");

OrderDetail OD = new OrderDetail();
SOQ.Detail.Add(OD);
//Various Order Detail properties can be added like warehouse , sales reps , userfields etc
OD.InventoryItem = new InventoryItem("ItemA"); //Use the inventoryItem constructor to specify a Item
OD.Quantity = 10;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 20;

SOQ.Save(); //You can Save the quote or process a existing quote in to a invoice

```

Quantity can be reserved on a Sales Order if the Order Entry Defaults | Sales Orders is set to reserve quantities. The order has to be saved and not processed to reserve stock.

```

SalesOrder SO = new SalesOrder();
SO.Customer = new Customer("Cust1");

OrderDetail OD = new OrderDetail();
SO.Detail.Add(OD);
//Various Order Detail properties can be added like warehouse , sales reps , userfields etc
OD.InventoryItem = new InventoryItem("ItemA"); //Use the inventoryItem constructor to specify a Item
OD.Quantity = 5;
OD.Reserved = 5; //specify quantity to reserve
OD.UnitSellingPrice = 20;

SO.Process(); // To reserve quantity the order must be saved not processed

```

The following example is on processing a existing sales order and specifying the quantity for each line.

```

SalesOrder SO = new SalesOrder("S000023"); //Specify the order number to process
foreach (OrderDetail NewOD in SO.Detail)
{
    NewOD.ToProcess = NewOD.Outstanding; //specify outstanding quantity to process for each line other changes can be made
}
SO.Process(); //Process the SalesOrder

```

Inventory documents can be populated in tax exclusive or tax inclusive mode using the TaxMode property on the OrderBase record. When creating a new Order record or loose-standing OrderDetail object, TaxMode will default to TaxExclusive. However, when using one of the OrderDetailCollection Add methods,

the detail object will default to whatever the current tax mode on the document is. Set the tax mode before setting the unit price.

```
SalesOrder sord = new SalesOrder();  
sord.Account = new Customer("ABC009");  
sord.TaxMode = TaxMode.Inclusive;
```

How to specify my own invoice number?

You will notice the inventory document classes contain multiple overloads of the Complete and Process methods. Those methods accepting a 'reference' parameter will use that reference as the document invoice number. If one of the other methods is used, or a blank reference parameter supplied, an invoice number will be generated. If number generation is disabled in inventory defaults, an exception will follow. The final reference number used - whether generated or supplied - will become the function's return value.

Retrieved from "https://developerzone.pastel.co.za/index.php?title=C_Sales_Orders&oldid=299"

-
- This page was last modified on 18 October 2016, at 15:58.